



**Universidad
Nacional
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXAMEN PARCIAL DE PYTHON

PROGRAMACIÓN ORIENTADA A OBJETOS, REQUESTS Y ARCHIVOS

Instrucciones

- Lee cuidadosamente cada consigna antes de programar.
- El examen tiene una duración de **2 horas**.
- Se evaluarán: clases, decoradores, manejo de archivos, requests, generadores y excepciones.
- No se permite el uso de librerías externas más allá de **requests** y librerías estándar.

1. Ejercicio 1 – Sistema de Productos y Descuentos (50 pts)

Se desea desarrollar un módulo para manejar productos en una tienda y aplicar descuentos automáticos.

1. (Comprensiones y lambda – 8 pts) A partir de la lista:

```
1 productos = [  
2     {"nombre": "Notebook", "precio": 850000, "categoria": "Tecnologa"},  
3     {"nombre": "Auriculares", "precio": 35000, "categoria": "Audio"},  
4     {"nombre": "Mouse", "precio": 15000, "categoria": "Accesorios"},  
5     {"nombre": "Monitor", "precio": 220000, "categoria": "Tecnologa"},  
6 ]
```

Usa:

- Una función `lambda` para aplicar un descuento del 10% a todos los productos de categoría "Tecnología".
- Una **comprensión de listas** para obtener los nombres de los productos cuyo precio final sea mayor a 100000.

—

2. (Clases – 12 pts) Crea una clase `Producto` con:

- Atributos: `nombre`, `precio`, `categoria`.
- Método `mostrar()` que devuelva: "Notebook - \$850000 (Tecnología)".

Luego, crea una lista de instancias a partir de los diccionarios iniciales.

—

3. (POO avanzada – 10 pts) Extiende el sistema con las clases:

- `ProductoComestible`: añade atributo `fecha_vencimiento`.
- `ProductoElectronico`: añade atributo `garantia`.

Implementa herencia y un método sobrescrito `info_extra()` que muestre la información adicional.

—

4. (Decoradores – 10 pts) Implementa un decorador `@verificar_stock` que impida ejecutar una función si la cantidad en stock es 0. Si no hay stock, debe lanzar una excepción `SinStockError`.

—

5. (Archivos y excepciones – 10 pts) Crea una función `registrar_ventas(lista_productos)` que:

- Guarde cada producto vendido en un archivo `ventas.txt` con formato CSV: `nombre,precio,categoria,fecha`.
- Use manejo de excepciones para errores de archivo.

2. Ejercicio 2 – API de Cotizaciones y Generadores (50 pts)

Se necesita un módulo que consulte la API de cotizaciones y procese los valores del dólar.

1. **(Requests – 10 pts)** Realiza una solicitud HTTP GET a la API pública: `https://api.bluelytics.com.ar/v2/latest` para obtener las cotizaciones del dólar oficial y blue. Guarda la respuesta en un archivo `dolar.json`. Incluye manejo de excepciones ante fallos de conexión o respuesta inválida.

—

2. **(Archivos y Excepciones – 10 pts)** Lee el archivo `dolar.json` y muestra por pantalla:

- El valor de compra y venta del dólar blue.
- El valor promedio entre ambos.

Maneja excepciones de archivo no encontrado y JSON inválido.

—

3. **(Generadores – 10 pts)** Implementa un generador `variaciones(cotizaciones)` que reciba una lista de valores de dólar y devuelva la variación entre cada par consecutivo (por ejemplo, $980 \rightarrow 995 = +15$).

—

4. **(Lambda y Comprensiones – 10 pts)** Usa:

- Una `lambda` para redondear las cotizaciones a 2 decimales.
- Una **comprensión de diccionarios** que asocie cada tipo de dólar con su valor de venta.

—

5. **(Clases y Decoradores – 10 pts)** Crea una clase `ReporteDolar` con:

- Atributos: `fecha`, `oficial`, `blue`.
- Método `resumen()` que imprima las cotizaciones y su diferencia.

Implementa un decorador `@mostrar_encabezado` que imprima: `Γ== REPORTE DIARIO DE COTIZACIONES ==` antes de ejecutar el método `resumen()`.